

P2A1 Árboles binarios de búsqueda

Preguntas y Respuestas

2. ¿Qué propiedad debe cumplir todo Árbol Binario de Búsqueda?

Para cada nodo, todas las claves de su subárbol izquierdo deben ser menores que su clave y todas las del subárbol derecho deben ser mayores. Esta regla se aplica también dentro de ambos subárboles; no basta con comparar únicamente los hijos inmediatos. En esta implementación no se insertan claves repetidas.

3. ¿Cuál es la raíz del árbol construido?

50, porque fue el primer valor insertado.

4. ¿Qué nodos son hojas?

20, 40, 60 y 80, porque no tienen hijos.

5. ¿Qué valores pertenecen al subárbol izquierdo de 50 y cuáles al derecho?

- Subárbol izquierdo: 30, 20 y 40.

- Subárbol derecho: 70, 60 y 80.

6. ¿Qué secuencia esperas obtener con el recorrido inorden?

20, 30, 40, 50, 60, 70, 80. El recorrido visita subárbol izquierdo, nodo actual y subárbol derecho.

7. Método de búsqueda

¿Por qué no es necesario recorrer todos los nodos del árbol para buscar una clave?

Porque el orden del ABB permite descartar uno de los dos subárboles en cada comparación. Si la clave es menor, solo se busca a la izquierda; si es mayor, solo a la derecha. Se recorre un camino de longitud proporcional a la altura. En un árbol equilibrado el costo es $O(\log n)$; en uno degenerado puede ser $O(n)$.

Si se busca 40, ¿qué nodos se visitan y en qué orden?

50 → 30 → 40. Desde 50 se va a la izquierda; desde 30, a la derecha; en 40 se encuentra la clave.

Si se busca 90, ¿qué condición permitirá concluir que no existe?

Se visita 50 → 70 → 80 y se intenta continuar hacia el hijo derecho de 80. Esa referencia es null, por lo que no existe un nodo con 90 en ese camino.

¿Qué valor booleano debe regresar el caso base cuando el nodo actual es null?

falso: un subárbol vacío no contiene la clave.

¿Qué ocurriría si el árbol no respetara la regla menor-izquierda y mayor-derecha?

La búsqueda podría descartar el subárbol que contiene la clave y devolver false aunque el valor sí exista. El algoritmo depende de que se cumpla esa regla.

8. Método de eliminación

¿Por qué la eliminación requiere más casos que la búsqueda?

Buscar solo determina si la clave existe. Eliminar también modifica los enlaces y debe conservar los demás nodos y el orden del ABB. La reconexión depende de si el nodo tiene cero, uno o dos hijos.

¿Qué debe ocurrir si la clave que se desea eliminar no existe?

El árbol debe conservarse. Al llegar a un subárbol vacío, se devuelve null; durante el regreso de la recursión se conservan los nodos existentes y sus enlaces.

¿Por qué eliminar un nodo hoja es el caso más sencillo?

Porque no tiene descendientes que deban reconectarse. Basta con devolver null para que su padre deje de apuntar a ese nodo. Si era la única raíz, el árbol queda vacío.

Si un nodo tiene solamente un hijo, ¿por qué puede devolverse directamente la referencia a ese hijo?

Porque ese hijo, junto con sus descendientes, ya pertenece al intervalo de claves permitido en esa posición. El padre puede apuntar directamente al hijo y así se conserva todo el subárbol restante.

¿Por qué el menor valor del subárbol derecho es un candidato adecuado para sustituir a un nodo con dos hijos?

Porque es el sucesor inorden: la menor clave que es mayor que la del nodo eliminado. Es mayor que todas las claves del subárbol izquierdo y, una vez eliminado de su ubicación original, menor que todas las claves restantes del subárbol derecho.

Después de copiar el valor sustituto, ¿por qué todavía es necesario eliminar ese valor de su ubicación original?

Porque copiar la clave no elimina el nodo que la contenía. Si se dejara en su ubicación original, la clave quedaría repetida y se incumpliría la regla estricta del ABB de esta implementación.

¿Qué riesgo existiría si se eliminara un nodo con dos hijos sin reconectar correctamente sus subárboles?

Podrían perderse referencias a nodos que deben conservarse, dejarse claves en una posición incorrecta o crearse enlaces inválidos. Por ejemplo, si el sucesor tiene un hijo derecho, ese hijo debe ocupar su posición al eliminarlo.

¿Por qué eliminar la raíz puede modificar la variable raíz del árbol?

Porque la referencia que representa la entrada al árbol puede cambiar. Si la raíz es hoja, la nueva referencia es null; si tiene un hijo, ese hijo se convierte en la nueva raíz. Por eso se escribe `raiz = eliminarRec(raiz, clave)`. En el caso de dos hijos usado aquí, se conserva el objeto de la raíz y se modifica su clave.

¿Qué propiedad debe seguir cumpliendo el árbol después de cualquier eliminación?

Todas las claves del subárbol izquierdo de cada nodo deben seguir siendo menores y todas las del derecho mayores. El inorden debe mostrar las claves restantes en orden ascendente, sin duplicados.

9. Método auxiliar: encontrar el valor mínimo

¿Hacia qué dirección debes desplazarte para encontrar el mínimo?

Hacia la izquierda.

¿Qué condición indica que ya encontraste el nodo mínimo?

Que el nodo actual no tenga hijo izquierdo: `actual.izquierdo == null`.

¿Cuál es el mínimo del subárbol cuya raíz es 70 en el árbol inicial?

60.

10. Reflexiones finales

¿Cómo ayuda el recorrido inorden a comprobar que el ABB conserva su estructura?

Permite comprobar el orden: si las claves aparecen en orden estrictamente ascendente, se respeta la ordenación del ABB, suponiendo que los enlaces formen un árbol válido. Además, comparar la lista con las claves esperadas ayuda a detectar pérdidas o duplicados. El inorden no demuestra que el árbol esté equilibrado ni que conserve exactamente su forma inicial.

Explica con tus palabras el caso de eliminación que consideraste más difícil.

El caso de dos hijos exige más cuidado porque ambos subárboles deben conservarse. Se busca el mínimo del subárbol derecho, se copia su clave al nodo encontrado y se elimina el sucesor de su ubicación original, reconectando cualquier hijo que tenga.

¿Qué papel cumple la recursividad en los métodos de búsqueda y eliminación?

Cada llamada resuelve el mismo problema sobre un subárbol más pequeño. En búsqueda, el resultado booleano regresa hasta la primera llamada. En eliminación, al regresar se actualizan los enlaces usando las referencias devueltas por cada llamada.

¿Qué aprendiste sobre el cambio de referencias entre nodos al eliminar elementos?

La idea central es que quitar un nodo requiere actualizar la referencia que apuntaba a él. La nueva referencia puede ser null, uno de sus hijos o el mismo nodo con una clave sustituta. Por eso no basta con llamar al método recursivo: hay que asignar su resultado al enlace correspondiente.

Si tuvieras que explicar a un compañero la diferencia entre buscar y eliminar en un ABB, ¿qué le dirías?

Buscar sigue las comparaciones y responde si la clave existe, sin cambiar el árbol. Eliminar primero localiza la clave y después modifica los enlaces o copia una clave sustituta para quitarla conservando el resto del árbol y la propiedad del ABB.